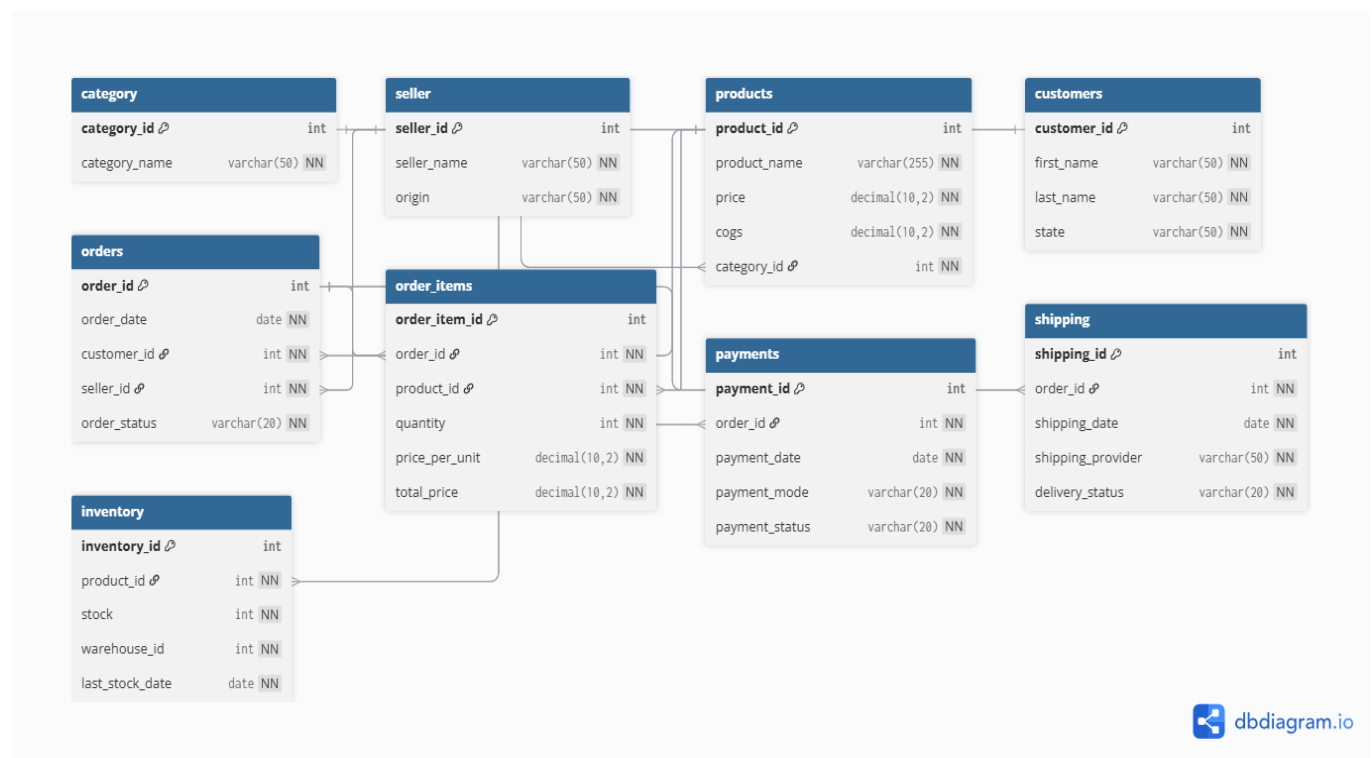


SQL Project Amazon Sales Analysis :-

- This project analyzes Amazon sales data from 2021 to 2024 to extract valuable business insights. The aim of this project is to showcase end-to-end SQL skills — from database design and querying, to advanced concepts like window functions, stored procedures, and CTEs.
- The project uses sample data generated with Python's Faker library to simulate real-world Amazon transactions.

Database Tables & Relationships :-



Agenda :-

Tech Stack :-

- Database: MySQL
 - Data Generation: Python (Faker library)
 - Tools: MySQL Workbench, VS Code, GitHub, dbdiagram.io
-

Beginner Concepts :-

- Database creation & table design
 - Data retrieval & filtering (SELECT, WHERE)
 - Aggregations (GROUP BY, HAVING, SUM, COUNT)
 - Multi-table JOINS
-

Intermediate Concepts :-

- Conditional logic (CASE WHEN, IF ELSE)
 - Date functions (YEAR, MONTH, QUARTER, DATEDIFF, DAYNAME, YEARWEEK)
 - Generating synthetic sales data with Python Faker
-

Advanced Concepts :-

- Window functions (RANK, DENSE_RANK, ROW_NUMBER / LEAD, LAG for growth trends)
- Common Table Expressions (CTEs, Recursive CTEs)
- Stored Procedures for automating sales & inventory updates
- Pivoting data for reports

Database Tables Schema :-

-- Create Amazon Sales Database

```
CREATE DATABASE amazon_sales;  
USE amazon_sales;
```

-- Category Table

```
CREATE TABLE category (  
    category_id INT PRIMARY KEY,  
    category_name VARCHAR(50) NOT NULL  
);
```

-- Seller Table

```
CREATE TABLE seller (  
    seller_id INT PRIMARY KEY,  
    seller_name VARCHAR(50) NOT NULL,  
    origin VARCHAR(50) NOT NULL  
);
```

-- Products Table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(255) NOT NULL,  
    price DECIMAL(10, 2) NOT NULL,  
    cogs DECIMAL(10, 2) NOT NULL,  
    category_id INT NOT NULL,  
    FOREIGN KEY (category_id) REFERENCES category(category_id)  
);
```

-- Customers Table

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    state VARCHAR(50) NOT NULL  
);
```

-- Orders Table

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  order_date DATE NOT NULL,  
  customer_id INT NOT NULL,  
  seller_id INT NOT NULL,  
  order_status VARCHAR(20) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),  
  FOREIGN KEY (seller_id) REFERENCES seller(seller_id)  
);
```

-- Order_items Table

```
CREATE TABLE order_items (  
  order_item_id INT PRIMARY KEY,  
  order_id INT NOT NULL,  
  product_id INT NOT NULL,  
  quantity INT NOT NULL,  
  price_per_unit DECIMAL(10, 2) NOT NULL,  
  total_price DECIMAL(10, 2) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

-- Payments Table

```
CREATE TABLE payments (  
  payment_id INT PRIMARY KEY,  
  order_id INT NOT NULL,  
  payment_date DATE NOT NULL,  
  payment_mode VARCHAR(20) NOT NULL,  
  payment_status VARCHAR(20) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

-- Shipping Table

```
CREATE TABLE shipping (  
    shipping_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    shipping_date DATE NOT NULL,  
    shipping_provider VARCHAR(50) NOT NULL,  
    delivery_status VARCHAR(20) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES orders(order_id)  
);
```

-- Inventory Table

```
CREATE TABLE inventory (  
    inventory_id INT PRIMARY KEY,  
    product_id INT NOT NULL,  
    stock INT NOT NULL,  
    warehouse_id INT NOT NULL,  
    last_stock_date DATE NOT NULL,  
    FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

1. List all customers from California.

```
SELECT *  
FROM customers  
WHERE state = 'California';
```

2. Show all products in the 'electronics' category.

```
SELECT c.category_name, p.product_name  
FROM category c  
JOIN products p ON c.category_id = p.category_id  
WHERE c.category_name = 'electronics';
```

3. Find all orders with a 'Cancelled' status.

```
SELECT *  
FROM orders  
WHERE order_status = 'Cancelled';
```

4. Count the total number of sellers from the 'USA'.

```
SELECT COUNT(*)  
FROM seller  
WHERE origin = 'USA';
```

5. Display the top 5 most expensive products.

```
SELECT *  
FROM products  
ORDER BY price DESC  
LIMIT 5;
```

6. Find all payments made via 'PayPal'.

```
SELECT *  
FROM payments  
WHERE payment_mode = 'PayPal';
```

7. List all orders placed in the year 2023.

```
SELECT *  
FROM orders  
WHERE YEAR(order_date) = 2023;
```

8. Show the product names and their categories.

```
SELECT p.product_name, c.category_name  
FROM products p  
JOIN category c ON p.category_id = c.category_id;
```

9. Find customers whose first name starts with 'A'.

```
SELECT *  
FROM customers  
WHERE first_name LIKE 'A%';
```

10. Count how many orders each customer has placed.

```
SELECT customer_id, COUNT(*) AS total_orders  
FROM orders  
GROUP BY customer_id;
```

11. Show all shipping records where the delivery status is 'Delivered'.

```
SELECT *  
FROM shipping  
WHERE delivery_status = 'Delivered';
```

12. Find the average price of all products.

```
SELECT ROUND(AVG(price), 2) AS avg_price  
FROM products;
```

13. List all orders along with the customer's first and last name.

```
SELECT o.*, c.first_name, c.last_name  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id;
```

14. Show products with a price between \$50 and \$100.

```
SELECT *  
FROM products  
WHERE price BETWEEN 50 AND 100;
```

15. Find the total number of items sold for each product.

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_items  
FROM order_items oi  
JOIN products p ON oi.product_id = p.product_id  
GROUP BY p.product_id, p.product_name;
```

16. List all sellers and their origin, ordered by origin alphabetically.

```
SELECT *  
FROM seller  
ORDER BY origin;
```

17. Show the earliest and latest order dates in the dataset.

```
SELECT MIN(order_date) AS earliest_order, MAX(order_date) AS last_order_date  
FROM orders;
```

18. Find all orders that were shipped by 'FedEx'.

```
SELECT o.*  
FROM orders o  
JOIN shipping s ON o.order_id = s.order_id  
WHERE shipping_provider = 'FedEx';
```

19. Count the number of products in each category.

```
SELECT c.category_name, COUNT(*) AS total_products  
FROM products p  
JOIN category c ON p.category_id = c.category_id  
GROUP BY c.category_name;
```

20. Show the total revenue generated from each order (from order_items).

```
SELECT order_id, SUM(total_price) AS total_revenue  
FROM order_items  
GROUP BY order_id;
```

21. List all orders that have a payment status of 'Failed'.

```
SELECT *  
FROM payments  
WHERE payment_status = 'Failed';
```

22. Find the customer who has placed the most orders.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) AS customer_name,  
       COUNT(o.order_id) AS total_orders  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
GROUP BY CONCAT(c.first_name, ' ', c.last_name)  
ORDER BY total_orders DESC  
LIMIT 1;
```

23. Show all products that are out of stock (stock = 0).

```
SELECT p.product_id, p.product_name  
FROM inventory i  
JOIN products p ON i.product_id = p.product_id  
WHERE i.stock = 0;
```

24. List the names of all shipping providers used.

```
SELECT DISTINCT shipping_provider
FROM shipping;
```

25. Find the average quantity of items ordered per order.

```
SELECT ROUND(AVG(quantity), 0) AS avg_quantity
FROM order_items;
```

26. Show all orders that are still 'Processing'.

```
SELECT *
FROM orders
WHERE order_status = 'Processing';
```

27. Count how many orders were delivered to each state.

```
SELECT c.state, COUNT(*) AS total_orders
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.state
ORDER BY total_orders DESC;
```

28. Find the most common payment mode.

```
SELECT payment_mode, COUNT(*) AS total_payments
FROM payments
GROUP BY payment_mode
ORDER BY total_payments DESC
LIMIT 1;
```

29. List all products with their seller's name.

```
SELECT p.product_id, p.product_name, s.seller_name
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
JOIN seller s ON s.seller_id = o.seller_id;
```

30. Show the total value of inventory in each warehouse.

```
SELECT i.warehouse_id,  
       COUNT(stock) AS total_stock,  
       SUM(p.price * i.stock) AS total_value  
FROM products p  
LEFT JOIN inventory i ON i.product_id = p.product_id  
GROUP BY i.warehouse_id;
```

31. Calculate the total sales revenue for each month in 2023.

```
SELECT MONTH(o.order_date) AS month_,  
       SUM(oi.total_price) AS total_revenue  
FROM order_items oi  
JOIN orders o ON oi.order_id = o.order_id  
WHERE YEAR(o.order_date) = 2023  
GROUP BY MONTH(o.order_date)  
ORDER BY MONTH(o.order_date);
```

32. Find the top 5 best-selling products by quantity sold.

```
SELECT p.product_id, p.product_name,  
       SUM(oi.quantity) AS total_quantity_sold  
FROM order_items oi  
JOIN products p ON oi.product_id = p.product_id  
GROUP BY p.product_id, p.product_name  
ORDER BY total_quantity_sold DESC  
LIMIT 5;
```

33. Calculate the average order value (AOV) for each customer.

```
SELECT c.customer_id,  
       CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
       ROUND(AVG(oi.total_price), 2) AS AOV  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN customers c ON o.customer_id = c.customer_id  
GROUP BY c.customer_id, CONCAT(c.first_name, ' ', c.last_name);
```

34. Show the monthly growth rate (%) in revenue for 2024.

```
WITH cte AS (  
    SELECT  
        MONTH(o.order_date) AS month_,  
        SUM(oi.total_price) AS current_revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    WHERE YEAR(o.order_date) = 2024  
    GROUP BY MONTH(o.order_date)  
    ORDER BY MONTH(o.order_date)  
)  
prev_revenue AS (  
    SELECT  
        month_,  
        current_revenue,  
        LAG(current_revenue, 1) OVER (ORDER BY month_) AS previous_revenue  
    FROM cte  
)  
SELECT  
    month_,  
    current_revenue,  
    previous_revenue,  
    ROUND(((current_revenue - previous_revenue) / previous_revenue) * 100, 2) AS  
growth_ratio_percent  
FROM prev_revenue;
```

35. Identify customers who have made more than 10 orders.

```
SELECT CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
       COUNT(o.order_id) AS total_orders  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
GROUP BY CONCAT(c.first_name, ' ', c.last_name)  
HAVING COUNT(o.order_id) > 10;
```

36. Find the category that generates the highest total revenue.

```
SELECT c.category_id, c.category_name,  
       SUM(oi.total_price) AS total_revenue  
FROM order_items oi  
JOIN products p ON oi.product_id = p.product_id  
JOIN category c ON p.category_id = c.category_id  
GROUP BY c.category_id, c.category_name  
ORDER BY total_revenue DESC  
LIMIT 1;
```

37. Calculate the average time between order date and shipping date for each shipping provider.

```
SELECT s.shipping_id, s.shipping_provider,  
       ROUND(AVG(DATEDIFF(s.shipping_date, o.order_date)), 0) AS avg_time  
FROM orders o  
JOIN shipping s ON o.order_id = s.order_id  
GROUP BY s.shipping_id, s.shipping_provider;
```

38. Show the top 3 states that generate the highest revenue.

```
SELECT c.state, SUM(oi.total_price) AS total_revenue  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
JOIN customers c ON o.customer_id = c.customer_id  
GROUP BY c.state  
ORDER BY total_revenue DESC  
LIMIT 3;
```

39. Identify products that have never been ordered.

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

40. Find the seller with the highest total sales revenue.

```
SELECT s.seller_id, s.seller_name,
       SUM(oi.total_price) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN seller s ON o.seller_id = s.seller_id
GROUP BY s.seller_id, s.seller_name
ORDER BY total_revenue DESC
LIMIT 1;
```

41. Calculate the percentage of orders that were returned.

```
SELECT
  SUM(CASE WHEN order_status = 'Returned' THEN 1 ELSE 0 END) AS total_returned_orders,
  ROUND(SUM(CASE WHEN order_status = 'Returned' THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS
  returned_orders_percentage
FROM orders;
```

42. Show the customer retention rate month-over-month.

```
WITH monthly_customers AS (  
    SELECT  
        MONTH(o.order_date) AS month_,  
        YEAR(o.order_date) AS year_,  
        COUNT(DISTINCT o.customer_id) AS unique_customers  
    FROM orders o  
    GROUP BY YEAR(o.order_date), MONTH(o.order_date)  
),  
previous_customers AS (  
    SELECT  
        year_,  
        month_,  
        unique_customers AS current_customers,  
        LAG(unique_customers, 1) OVER (ORDER BY year_, month_) AS previous_customers  
    FROM monthly_customers  
)  
SELECT  
    year_,  
    month_,  
    current_customers,  
    previous_customers,  
    ROUND((current_customers / previous_customers) * 100, 2) AS retention_rate  
FROM previous_customers  
ORDER BY year_, month_;
```

43. Find the day of the week with the highest number of orders.

```
WITH daywise_orders AS (  
    SELECT  
        YEAR(o.order_date) AS year_,  
        DAYNAME(o.order_date) AS day_name,  
        COUNT(o.order_id) AS total_orders  
    FROM orders o  
    GROUP BY YEAR(o.order_date), DAYNAME(o.order_date)  
)  
,  
ranked_orders AS (  
    SELECT  
        year_,  
        day_name,  
        total_orders,  
        RANK() OVER (PARTITION BY year_ ORDER BY total_orders DESC) AS rnk  
    FROM daywise_orders  
)  
SELECT year_, day_name, total_orders  
FROM ranked_orders  
WHERE rnk = 1;
```

44. Identify orders where the payment was made after the order was shipped.

```
WITH order_payment AS (  
    SELECT  
        o.order_id,  
        o.order_date,  
        s.shipping_date,  
        p.payment_date,  
        p.payment_mode,  
        p.payment_status  
    FROM orders o  
    JOIN shipping s ON o.order_id = s.order_id  
    JOIN payments p ON o.order_id = p.order_id  
)  
SELECT  
    order_id,  
    order_date,  
    shipping_date,  
    payment_date,  
    payment_mode,  
    payment_status  
FROM order_payment  
WHERE payment_date > shipping_date;
```

45. Calculate the profit margin for each product and categorize it.

```
WITH margin AS (  
    SELECT  
        p.product_id,  
        p.product_name,  
        ROUND(((oi.price_per_unit - p.cogs) / oi.price_per_unit) * 100, 2) AS profit_margin  
    FROM products p  
    JOIN order_items oi ON p.product_id = oi.product_id  
)  
SELECT  
    product_id,  
    product_name,  
    profit_margin,  
    CASE  
        WHEN profit_margin > 50 THEN 'High'  
        WHEN profit_margin BETWEEN 20 AND 50 THEN 'Medium'  
        ELSE 'Low'  
    END AS profit_margin_category  
FROM margin;
```

46. Show the running total of revenue for each month in 2023.

```
SELECT YEAR(o.order_date) AS year_,  
    MONTH(o.order_date) AS month_,  
    SUM(oi.total_price) AS monthly_revenue,  
    SUM(SUM(oi.total_price)) OVER (ORDER BY MONTH(o.order_date)) AS running_total  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
WHERE YEAR(o.order_date) = 2023  
GROUP BY YEAR(o.order_date), MONTH(o.order_date)  
ORDER BY YEAR(o.order_date), MONTH(o.order_date);
```

47. Identify the most valuable customer (by total spend) for each state.

```
WITH customer_revenue AS (  
    SELECT  
        c.customer_id,  
        CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
        c.state,  
        SUM(oi.total_price) AS total_revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, CONCAT(c.first_name, ' ', c.last_name), c.state  
)  
ranked AS (  
    SELECT  
        customer_id,  
        full_name,  
        state,  
        total_revenue,  
        DENSE_RANK() OVER (PARTITION BY state ORDER BY total_revenue DESC) AS rnk  
    FROM customer_revenue  
)  
SELECT  
    customer_id,  
    full_name,  
    state,  
    total_revenue  
FROM ranked  
WHERE rnk = 1;
```

48. Calculate the average shipping time for delivered orders.

```
SELECT
    ROUND(AVG(DATEDIFF(s.shipping_date, o.order_date)), 2) AS avg_shipping_time
FROM orders o
JOIN shipping s ON o.order_id = s.order_id
WHERE s.delivery_status = 'Delivered';
```

49. Show the top 5 customers with the highest average order value.

```
SELECT c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    ROUND(AVG(oi.total_price), 2) AS avg_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, CONCAT(c.first_name, ' ', c.last_name)
ORDER BY avg_revenue DESC
LIMIT 5;
```

50. Find sellers who sell products in more than 3 different categories.

```
SELECT
    s.seller_id,
    s.seller_name,
    COUNT(DISTINCT c.category_id) AS total_categories
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN seller s ON o.seller_id = s.seller_id
JOIN products p ON oi.product_id = p.product_id
JOIN category c ON p.category_id = c.category_id
GROUP BY s.seller_id, s.seller_name
HAVING COUNT(DISTINCT c.category_id) > 3;
```

51. Calculate the refund rate by payment mode.

```
SELECT
    p.payment_mode,
    ROUND(
        (SUM(CASE WHEN p.payment_status = 'Failed' THEN 1 ELSE 0 END)
        + SUM(CASE WHEN o.order_status = 'Returned' THEN 1 ELSE 0 END))
        * 100.0 / COUNT(*), 2
    ) AS refund_rate_percentage
FROM payments p
JOIN orders o ON p.order_id = o.order_id
GROUP BY p.payment_mode;
```

52. Identify the peak sales hour of the day.

```
SELECT
    HOUR(order_time) AS hour_,
    COUNT(*) AS total_orders
FROM orders
GROUP BY HOUR(order_time)
ORDER BY total_orders DESC
LIMIT 1;
```

53. Show the revenue generated by each category for each quarter of 2023.

```
SELECT c.category_name,
    QUARTER(o.order_date) AS quarter_,
    SUM(oi.total_price) AS total_revenue
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN category c ON p.category_id = c.category_id
WHERE YEAR(o.order_date) = 2023
GROUP BY c.category_name, QUARTER(o.order_date)
ORDER BY c.category_name, QUARTER(o.order_date);
```

54. Find customers who haven't placed an order in the last 6 months.

```
WITH max_date AS (  
    SELECT MAX(order_date) AS latest_order_date  
    FROM orders  
)  
customer_last_order AS (  
    SELECT  
        o.customer_id,  
        MAX(o.order_date) AS last_order_date  
    FROM orders o  
    GROUP BY o.customer_id  
)  
SELECT  
    c.customer_id,  
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
    clo.last_order_date  
FROM customers c  
JOIN customer_last_order clo ON c.customer_id = clo.customer_id  
JOIN max_date m ON 1=1  
WHERE clo.last_order_date < DATE_SUB(m.latest_order_date, INTERVAL 6 MONTH);
```

55. Calculate the stock turnover ratio for each product.

```
WITH sales AS (  
    SELECT  
        p.product_id,  
        p.product_name,  
        SUM(oi.quantity) AS total_units_sold  
    FROM order_items oi  
    JOIN products p ON oi.product_id = p.product_id  
    GROUP BY p.product_id, p.product_name  
)  
,  
inventory_avg AS (  
    SELECT  
        i.product_id,  
        AVG(i.stock) AS avg_inventory  
    FROM inventory i  
    GROUP BY i.product_id  
)  
SELECT  
    s.product_id,  
    s.product_name,  
    s.total_units_sold,  
    ia.avg_inventory,  
    ROUND(s.total_units_sold / NULLIF(ia.avg_inventory, 0), 2) AS stock_turnover_ratio  
FROM sales s  
JOIN inventory_avg ia ON s.product_id = ia.product_id;
```

56. Show the YoY growth in customer acquisition.

```
WITH yearly_customers AS (  
    SELECT  
        YEAR(order_date) AS year_,  
        COUNT(DISTINCT customer_id) AS new_customers  
    FROM orders  
    GROUP BY year_  
)  
,  
growth AS (  
    SELECT  
        year_,  
        new_customers,  
        LAG(new_customers, 1) OVER (ORDER BY year_) AS prev_year_customers  
    FROM yearly_customers  
)  
SELECT  
    year_,  
    new_customers,  
    prev_year_customers,  
    ROUND(((new_customers - prev_year_customers) / prev_year_customers) * 100, 2) AS  
yoy_growth_percentage  
FROM growth  
ORDER BY year_;
```

57. Identify orders where the item quantity is greater than 3.

```
SELECT o.*, oi.quantity  
FROM orders o  
JOIN order_items oi ON o.order_id = oi.order_id  
WHERE oi.quantity > 3;
```

58. Find the most common product category purchased by customers in each state.

```
WITH category_count AS (  
    SELECT  
        c.state,  
        ca.category_name,  
        COUNT(*) AS total_orders  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN products p ON oi.product_id = p.product_id  
    JOIN customers c ON o.customer_id = c.customer_id  
    JOIN category ca ON p.category_id = ca.category_id  
    GROUP BY c.state, ca.category_name  
)  
,  
ranked AS (  
    SELECT  
        state,  
        category_name,  
        total_orders,  
        DENSE_RANK() OVER (PARTITION BY state ORDER BY total_orders DESC) AS rnk  
    FROM category_count  
)  
SELECT state, category_name, total_orders  
FROM ranked  
WHERE rnk = 1;
```

59. Rank sellers within each category based on their total revenue.

```
WITH category_wise_revenue AS (  
    SELECT s.seller_name, c.category_name,  
           SUM(oi.total_price) AS total_revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN seller s ON o.seller_id = s.seller_id  
    JOIN products p ON p.product_id = oi.product_id  
    JOIN category c ON p.category_id = c.category_id  
    GROUP BY s.seller_name, c.category_name  
)  
  
ranked AS (  
    SELECT seller_name, category_name, total_revenue,  
           DENSE_RANK() OVER (PARTITION BY category_name ORDER BY total_revenue DESC) AS rnk  
    FROM category_wise_revenue  
)  
  
SELECT seller_name, category_name, total_revenue  
FROM ranked  
WHERE rnk = 1  
ORDER BY category_name;
```

60. Revenue by Category

- Find the revenue generated by each category and calculate its percentage contribution to total revenue.

```
WITH cte AS (  
    SELECT  
        c.category_id,  
        c.category_name,  
        SUM(ot.total_price) AS total_category_revenue  
    FROM order_items ot  
    JOIN products p ON ot.product_id = p.product_id  
    JOIN category c ON p.category_id = c.category_id  
    GROUP BY c.category_id, c.category_name  
)  
SELECT  
    category_id,  
    category_name,  
    total_category_revenue,  
    ROUND(  
        total_category_revenue / (SELECT SUM(total_price) FROM order_items) * 100,  
        2  
    ) AS contribute_percent  
FROM cte;
```

61. Average Order Value (AOV) per Customer

- Calculate the average order value for each customer who has placed more than 10 orders.

-
- Method 1: Based on Total Value / Total Orders

```
WITH cte AS (  
  SELECT  
    c.customer_id,  
    c.first_name,  
    c.last_name,  
    SUM(ot.total_price) AS total_value,  
    COUNT(o.order_id) AS total_orders  
  FROM orders o  
  JOIN customers c ON o.customer_id = c.customer_id  
  JOIN order_items ot ON o.order_id = ot.order_id  
  GROUP BY c.customer_id, c.first_name  
)  
SELECT  
  customer_id,  
  first_name,  
  last_name,  
  ROUND((total_value / total_orders), 2) AS AOV,  
  total_orders  
FROM cte  
WHERE total_orders >= 10;
```

- Method 2: Based on Average Order Items

```
WITH cte AS (  
    SELECT  
        c.customer_id,  
        c.first_name,  
        c.last_name,  
        ROUND(AVG(ot.total_price), 2) AS AOV,  
        COUNT(o.order_id) AS total_orders  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    JOIN order_items ot ON o.order_id = ot.order_id  
    GROUP BY c.customer_id, c.first_name  
)  
SELECT  
    customer_id,  
    first_name,  
    last_name,  
    AOV,  
    total_orders  
FROM cte  
WHERE total_orders >= 10;
```

62. Monthly Sales Trend

- Show monthly total sales and calculate growth compared to the previous month.

```
WITH monthly_sales AS (  
    SELECT  
        YEAR(o.order_date) AS year_,  
        MONTH(o.order_date) AS month_,  
        SUM(ot.quantity) AS total_units,  
        SUM(ot.total_price) AS current_sale  
    FROM orders o  
    JOIN order_items ot ON o.order_id = ot.order_id  
    GROUP BY YEAR(o.order_date), MONTH(o.order_date)  
)  
cte AS (  
    SELECT  
        year_,  
        month_,  
        total_units,  
        current_sale,  
        LAG(current_sale, 1) OVER (ORDER BY year_, month_) AS previous_sale  
    FROM monthly_sales  
)  
SELECT  
    year_,  
    month_,  
    current_sale,  
    previous_sale,  
    ROUND(  
        ((current_sale - previous_sale) / NULLIF(previous_sale, 0)) * 100,  
        2  
    ) AS monthly_growth  
FROM cte  
ORDER BY year_, month_;
```

63. Customers Without Orders

- List customers who registered but never placed an order.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

64. Best-Selling Categories by State

- Identify the best-selling product category in each state.

```
WITH state_category_sales AS (
    SELECT
        c.state,
        ca.category_name,
        SUM(ot.quantity) AS total_units
    FROM orders o
    JOIN order_items ot ON o.order_id = ot.order_id
    JOIN customers c ON o.customer_id = c.customer_id
    JOIN products p ON ot.product_id = p.product_id
    JOIN category ca ON p.category_id = ca.category_id
    GROUP BY c.state, ca.category_name
),
ranked AS (
    SELECT
        state,
        category_name,
        total_units,
        DENSE_RANK() OVER (PARTITION BY state ORDER BY total_units DESC) AS rnk
    FROM state_category_sales
)
```

```
SELECT
    state,
    category_name AS best_selling_category,
    total_units
FROM ranked
WHERE rnk = 1
ORDER BY state;
```

65. Customer Lifetime Value

- Calculate total units and revenue generated by each customer across their lifetime.

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    SUM(ot.quantity) AS total_units,
    SUM(ot.total_price) AS total_price
FROM orders o
JOIN order_items ot ON o.order_id = ot.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, full_name
ORDER BY c.customer_id;
```

66. Inventory Stock Alerts

- List products with stock level below 300 units.

```
SELECT
    p.product_id,
    p.product_name,
    i.stock,
    i.warehouse_id,
    i.last_stock_date
FROM inventory i
JOIN products p ON i.product_id = p.product_id
WHERE i.stock < 300;
```

67. Shipping Delays

- Identify orders where shipping took more than 7 days after order date.

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    o.customer_id,
    o.order_date,
    s.shipping_date,
    s.shipping_provider
FROM orders o
JOIN shipping s ON o.order_id = s.order_id
JOIN customers c ON o.customer_id = c.customer_id
WHERE DATEDIFF(s.shipping_date, o.order_date) > 7;
```

68. Top Performing Sellers

- Find the top 5 sellers based on total sales value.

```
SELECT
    s.seller_id,
    s.seller_name,
    s.origin,
    SUM(oi.total_price) AS total_sales_value
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN seller s ON o.seller_id = s.seller_id
GROUP BY s.seller_id, s.seller_name, s.origin
ORDER BY total_sales_value DESC
LIMIT 5;
```

69.Payment Success Rate

Question - 1

- Calculate percentage of successful payments across all orders.

```
SELECT
    p.payment_status,
    COUNT(*) AS total_count,
    ROUND(COUNT(*) / (SELECT COUNT(*) FROM payments) * 100, 2) AS payment_rate
FROM payments p
JOIN orders o ON p.order_id = o.order_id
GROUP BY p.payment_status;
```

Question - 2

- Success rate by payment mode.

```
SELECT
    p.payment_mode,
    p.payment_status,
    COUNT(*) AS total_payments,
    ROUND(
        COUNT(*) * 100.0 / (SELECT COUNT(*) FROM payments),
        2
    ) AS payment_rate_percent
FROM payments p
JOIN orders o ON p.order_id = o.order_id
GROUP BY p.payment_mode, p.payment_status
ORDER BY p.payment_mode;
```

70. Seller Orders by Status

- Count seller orders by different statuses (Delivered, Cancelled, etc.).

```
SELECT
  s.seller_id,
  s.seller_name,
  s.origin,
  o.order_status,
  SUM(oi.quantity) AS total_count
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN seller s ON o.seller_id = s.seller_id
GROUP BY s.seller_id, s.seller_name, s.origin, o.order_status
ORDER BY s.seller_id, total_count DESC;
```

71. Seller Order Status Ratio

- Show the ratio of each order status compared to seller's total orders.

```
WITH seller_status AS (  
    SELECT  
        s.seller_id,  
        s.seller_name,  
        s.origin,  
        o.order_status,  
        COUNT(o.order_id) AS status_count  
    FROM orders o  
    JOIN seller s ON o.seller_id = s.seller_id  
    GROUP BY s.seller_id, s.seller_name, s.origin, o.order_status  
)  
seller_totals AS (  
    SELECT  
        seller_id,  
        SUM(status_count) AS total_orders  
    FROM seller_status  
    GROUP BY seller_id  
)  
SELECT  
    ss.seller_id,  
    ss.seller_name,  
    ss.origin,  
    ss.order_status,  
    ss.status_count,  
    ROUND((ss.status_count / st.total_orders) * 100, 2) AS status_ratio_percent  
FROM seller_status ss  
JOIN seller_totals st ON ss.seller_id = st.seller_id  
ORDER BY ss.seller_id, status_ratio_percent DESC;
```

72. Pivot Orders by Status

- Show seller orders pivoted by status.

```
SELECT
    s.seller_id,
    s.seller_name,
    SUM(CASE WHEN o.order_status = 'Delivered' THEN oi.quantity ELSE 0 END) AS successful_orders,
    SUM(CASE WHEN o.order_status = 'Cancelled' THEN oi.quantity ELSE 0 END) AS cancelled_orders,
    SUM(CASE WHEN o.order_status = 'Returned' THEN oi.quantity ELSE 0 END) AS returned_orders,
    SUM(CASE WHEN o.order_status = 'Shipped' THEN oi.quantity ELSE 0 END) AS shipped_orders,
    SUM(CASE WHEN o.order_status = 'Processing' THEN oi.quantity ELSE 0 END) AS
processing_orders,
    SUM(CASE WHEN o.order_status = 'Pending' THEN oi.quantity ELSE 0 END) AS pending_orders,
    SUM(oi.quantity) AS total_orders
FROM seller s
JOIN orders o ON s.seller_id = o.seller_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY s.seller_id, s.seller_name;
```

73. Product Profit Margin

- Calculate profit margin for each product.

```
SELECT
    p.product_name,
    SUM(oi.quantity * oi.price_per_unit) AS total_revenue,
    SUM(oi.quantity * p.cogs) AS total_cost,
    SUM(oi.quantity * oi.price_per_unit) - SUM(oi.quantity * p.cogs) AS profit_margin
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY profit_margin DESC;
```

74. Most Returned Products

- Find the top 10 most returned products.

```
WITH cte AS (  
    SELECT  
        p.product_id,  
        p.product_name,  
        SUM(oi.quantity) AS total_units_sold,  
        SUM(CASE WHEN o.order_status = 'Returned' THEN oi.quantity ELSE 0 END) AS returned_units  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN products p ON oi.product_id = p.product_id  
    GROUP BY p.product_id, p.product_name  
    ORDER BY returned_units DESC  
    LIMIT 10  
)  
SELECT  
    product_id,  
    product_name,  
    total_units_sold,  
    returned_units,  
    ROUND((returned_units / total_units_sold) * 100, 2) AS return_percent  
FROM cte;
```

75. Inactive Sellers

- Find sellers with no sales in the last 2 years.

```
SELECT *
FROM seller s
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.seller_id = s.seller_id
        AND o.order_date >= CURDATE() - INTERVAL 2 YEAR
);
```

76. Returning Customers :- Identify customers who returned more than 5 orders.

```
WITH cte AS (
    SELECT
        c.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS full_name,
        COUNT(o.order_id) AS total_orders,
        SUM(CASE WHEN o.order_status = 'Returned' THEN 1 ELSE 0 END) AS returned_orders
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY c.customer_id, full_name
    ORDER BY returned_orders DESC
)
SELECT
    customer_id,
    full_name,
    total_orders,
    returned_orders,
    CASE
        WHEN returned_orders > 5 THEN 'Returning_customer'
        ELSE 'Normal_Customer'
    END AS customer_category
FROM cte;
```

77. Top 5 Customers by Sales (per State)

```
WITH cte AS (  
    SELECT  
        c.customer_id,  
        CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
        c.state,  
        COUNT(o.order_id) AS total_orders,  
        SUM(oi.total_price) AS total_price  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, full_name, c.state  
)  
ranked AS (  
    SELECT  
        customer_id,  
        full_name,  
        state,  
        total_orders,  
        total_price,  
        DENSE_RANK() OVER(PARTITION BY state ORDER BY total_price DESC) AS rnk  
    FROM cte  
)  
SELECT  
    customer_id,  
    full_name,  
    state,  
    total_orders,  
    total_price  
FROM ranked  
WHERE rnk <= 5  
ORDER BY state, rnk, total_price DESC;
```

78. Top 5 Customers by Orders (per State)

```
WITH cte AS (  
    SELECT  
        c.customer_id,  
        CONCAT(c.first_name,' ',c.last_name) AS full_name,  
        c.state,  
        COUNT(o.order_id) AS total_orders  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, full_name, c.state  
)  
ranked AS (  
    SELECT  
        customer_id,  
        full_name,  
        state,  
        total_orders,  
        DENSE_RANK() OVER(PARTITION BY state ORDER BY total_orders DESC) AS rnk  
    FROM cte  
)  
SELECT  
    customer_id,  
    full_name,  
    state,  
    total_orders  
FROM ranked  
WHERE rnk <= 5  
ORDER BY state, rnk, total_orders DESC;
```

79. Revenue by Shipping Provider

- Calculate total revenue, total orders, and average delivery time per shipping provider.

```
SELECT
    s.shipping_provider,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(SUM(oi.total_price), 2) AS total_revenue,
    ROUND(AVG(DATEDIFF(s.shipping_date, o.order_date)), 0) AS avg_delivery_time_days
FROM orders o
JOIN shipping s ON o.order_id = s.order_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY s.shipping_provider
ORDER BY total_revenue DESC;
```

80. Top 10 Products by Growth (2023 → 2024)

```
WITH cte AS (
    SELECT
        p.product_id,
        p.product_name,
        c.category_name,
        YEAR(o.order_date) AS year,
        SUM(oi.total_price) AS total_revenue
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    JOIN products p ON oi.product_id = p.product_id
    JOIN category c ON p.category_id = c.category_id
    WHERE YEAR(o.order_date) IN (2023, 2024)
    GROUP BY p.product_id, p.product_name, c.category_name, YEAR(o.order_date)
),
year_wise_revenue AS (
    SELECT
        product_id,
        product_name,
```

```

        category_name,
        SUM(CASE WHEN year = 2023 THEN total_revenue ELSE 0 END) AS 2023_revenue,
        SUM(CASE WHEN year = 2024 THEN total_revenue ELSE 0 END) AS 2024_revenue
    FROM cte
    GROUP BY product_id, product_name, category_name
),
growth_ratio AS (
    SELECT
        product_id,
        product_name,
        category_name,
        2023_revenue,
        2024_revenue,
        (2024_revenue - 2023_revenue) AS revenue_diff,
        ROUND(((2024_revenue - 2023_revenue) / 2023_revenue), 2) AS growth_pattern
    FROM year_wise_revenue
),
ranked_ratio AS (
    SELECT
        product_id,
        product_name,
        category_name,
        2023_revenue,
        2024_revenue,
        revenue_diff,
        growth_pattern,
        DENSE_RANK() OVER(ORDER BY growth_pattern DESC) AS rnk
    FROM growth_ratio
)
SELECT
    product_id,

```

```

product_name,
category_name,
2023_revenue,
2024_revenue,
revenue_diff,
CONCAT(growth_pattern, '%') AS growth_percent
FROM ranked_ratio
WHERE rnk <= 10;

```

81. Stored Procedure: Update Inventory after Sale

- When a product is sold, update inventory automatically by reducing sold quantity.

```

DELIMITER $$
CREATE PROCEDURE update_inventory_stock_data(
    IN p_customer_id INT,
    IN p_product_id INT,
    IN p_order_item_id INT,
    IN p_order_id INT,
    IN p_quantity INT,
    IN p_seller_id INT
)
BEGIN
    DECLARE v_count INT;
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_product_name VARCHAR(50);

    SELECT price, product_name
    INTO v_price, v_product_name
    FROM products
    WHERE product_id = p_product_id;

    SELECT COUNT(*)
    INTO v_count
    FROM inventory

```

```

WHERE product_id = p_product_id AND stock >= p_quantity;

IF v_count > 0 THEN

    INSERT INTO orders(order_id, order_date, customer_id, seller_id)

    VALUES (p_order_id, CURDATE(), p_customer_id, p_seller_id);

    INSERT INTO order_items(order_item_id, order_id, product_id, quantity, price_per_unit,
total_price)

    VALUES (p_order_item_id, p_order_id, p_product_id, p_quantity, v_price, p_quantity * v_price);

    UPDATE inventory

    SET stock = stock - p_quantity

    WHERE product_id = p_product_id;

    SELECT CONCAT('Product sale added. Inventory updated for: ', v_product_name) AS message;

ELSE

    SELECT CONCAT('Product ', v_product_name, ' is not available in required quantity.') AS
message;

END IF;

END$$

DELIMITER ;

```

82.Perform a cohort analysis to calculate the retention rate of customers based on the month

```

WITH first_purchase AS (

    -- Find each customer's first purchase month

    SELECT

        customer_id,

        MIN(DATE_FORMAT(order_date, '%Y-%m')) AS first_purchase_month

    FROM orders

    GROUP BY customer_id

),

cohort_activity AS (

    -- Track each customer's activity month

    SELECT

        o.customer_id,

        DATE_FORMAT(fp.first_purchase_month, '%Y-%m') AS cohort_month,

        DATE_FORMAT(o.order_date, '%Y-%m') AS activity_month

```

```

FROM orders o

JOIN first_purchase fp
  ON o.customer_id = fp.customer_id
),
retention AS (
  -- Count unique customers in each cohort and activity month
  SELECT
    cohort_month,
    activity_month,
    COUNT(DISTINCT customer_id) AS active_customers
  FROM cohort_activity
  GROUP BY cohort_month, activity_month
),
cohort_size AS (
  -- Total customers in each cohort (denominator for retention rate)
  SELECT
    cohort_month,
    COUNT(DISTINCT customer_id) AS cohort_customers
  FROM first_purchase
  GROUP BY cohort_month
)
SELECT
  r.cohort_month,
  r.activity_month,
  r.active_customers,
  c.cohort_customers,
  ROUND((r.active_customers * 100.0 / c.cohort_customers), 2) AS retention_rate
FROM retention r
JOIN cohort_size c
  ON r.cohort_month = c.cohort_month
ORDER BY r.cohort_month, r.activity_month;

```

83. Identify products that are frequently bought together (using order_id).

```
SELECT
    oi1.product_id AS product_id_1,
    p1.product_name AS product_name_1,
    oi2.product_id AS product_id_2,
    p2.product_name AS product_name_2,
    COUNT(*) AS frequency
FROM order_items oi1
JOIN order_items oi2
    ON oi1.order_id = oi2.order_id
    AND oi1.product_id < oi2.product_id
JOIN products p1 ON oi1.product_id = p1.product_id
JOIN products p2 ON oi2.product_id = p2.product_id
GROUP BY oi1.product_id, p1.product_name, oi2.product_id, p2.product_name
HAVING COUNT(*) > 1
ORDER BY frequency DESC
LIMIT 10;
```

84. Calculate the customer lifetime value (CLV) for each customer (total spend / tenure in months).

```
WITH customer_summary AS (  
    SELECT  
        c.customer_id,  
        CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
        MIN(o.order_date) AS first_order_date,  
        MAX(o.order_date) AS last_order_date,  
        SUM(oi.total_price) AS total_spend  
    FROM customers c  
    JOIN orders o ON c.customer_id = o.customer_id  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY c.customer_id, CONCAT(c.first_name, ' ', c.last_name)  
)  
clv_calc AS (  
    SELECT  
        customer_id,  
        full_name,  
        total_spend,  
        TIMESTAMPDIFF(MONTH, first_order_date, last_order_date) AS tenure_months  
    FROM customer_summary  
)  
SELECT  
    customer_id,  
    full_name,  
    total_spend,  
    CASE  
        WHEN tenure_months = 0 THEN ROUND(total_spend / 1, 2)  
        ELSE ROUND(total_spend / tenure_months, 2)  
    END AS clv  
FROM clv_calc  
ORDER BY clv DESC;
```

85. Calculate the 7-day rolling average of daily revenue for December 2023.

```
WITH cte AS (  
    SELECT  
        o.order_date,  
        SUM(oi.total_price) AS revenue  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    WHERE YEAR(o.order_date) = 2023  
        AND MONTH(o.order_date) = 12  
    GROUP BY o.order_date  
    ORDER BY o.order_date  
)  
SELECT  
    order_date,  
    revenue,  
    ROUND(AVG(revenue) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND CURRENT  
ROW),2) AS rolling_7day_avg  
FROM cte;
```

86. Identify customers who upgraded their average order value from one year to the next (e.g., 2022 vs 2023).

```
WITH cte AS (  
  SELECT  
    c.customer_id,  
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
    YEAR(o.order_date) AS year_,  
    ROUND(AVG(oi.total_price), 2) AS avg_order_value  
  FROM orders o  
  JOIN order_items oi ON o.order_id = oi.order_id  
  JOIN customers c ON o.customer_id = c.customer_id  
  WHERE YEAR(o.order_date) IN (2022, 2023)  
  GROUP BY c.customer_id, full_name, YEAR(o.order_date)  
)  
pivot AS (  
  SELECT  
    customer_id,  
    full_name,  
    MAX(CASE WHEN year_ = 2022 THEN avg_order_value END) AS `2022_revenue`,  
    MAX(CASE WHEN year_ = 2023 THEN avg_order_value END) AS `2023_revenue`  
  FROM cte  
  GROUP BY customer_id, full_name  
)  
SELECT  
  customer_id,  
  full_name,  
  `2022_revenue`,  
  `2023_revenue`  
FROM pivot  
WHERE `2023_revenue` > `2022_revenue`;
```

87. For each product, calculate the days when it was the top-selling product in its category.

```
WITH daily_sales AS (  
    SELECT  
        ca.category_name,  
        p.product_name,  
        o.order_date,  
        SUM(oi.quantity) AS total_units  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN products p ON oi.product_id = p.product_id  
    JOIN category ca ON p.category_id = ca.category_id  
    GROUP BY ca.category_name, p.product_name, o.order_date  
) ,  
ranked AS (  
    SELECT  
        category_name,  
        product_name,  
        order_date,  
        total_units,  
        DENSE_RANK() OVER (  
            PARTITION BY category_name, order_date  
            ORDER BY total_units DESC  
        ) AS rnk  
    FROM daily_sales  
)  
SELECT  
    category_name,  
    product_name,  
    order_date,  
    total_units  
FROM ranked  
WHERE rnk = 1  
ORDER BY category_name, order_date, product_name;
```

88. Use a recursive CTE to create a date dimension table and join it to analyze daily sales, even for days with no orders. Recursive CTE to generate a date dimension

```
SET SESSION cte_max_recursion_depth = 5000;

WITH RECURSIVE calender AS (
    SELECT MIN(order_date) AS date_
    FROM orders
    UNION ALL
    SELECT DATE_ADD(date_, INTERVAL 1 DAY)
    FROM calender
    WHERE date_ < (SELECT MAX(order_date) FROM orders)
),
daily_sales AS (
    SELECT
        o.order_date,
        SUM(oi.total_price) AS total_sales
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY o.order_date
)
SELECT
    c.date_ AS order_date,
    COALESCE(ds.total_sales, 0) AS total_sales
FROM calender c
LEFT JOIN daily_sales ds
    ON c.date_ = ds.order_date
ORDER BY order_date;
```

89. Identify the "next product" a customer buys after purchasing a specific product (e.g., 'Wireless Earbuds')

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,
    p.product_name,
    LEAD(p.product_name, 1) OVER (
        PARTITION BY c.customer_id ORDER BY o.order_date
    ) AS next_product
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN customers c ON o.customer_id = c.customer_id
```

90. Predict potential stock-outs: Identify products where the current stock is less than the average weekly sales volume.

```
WITH weekly_sales AS (
    SELECT
        oi.product_id,
        SUM(oi.quantity) / COUNT(DISTINCT YEARWEEK(o.order_date)) AS avg_weekly_sales
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    GROUP BY oi.product_id
)
SELECT
    p.product_id,
    p.product_name,
    i.stock,
    ROUND(w.avg_weekly_sales, 2) AS avg_weekly_sales
FROM products p
JOIN inventory i ON p.product_id = i.product_id
JOIN weekly_sales w ON p.product_id = w.product_id
WHERE i.stock < w.avg_weekly_sales;
```

91. Calculate the median order value for the entire dataset (without using MEDIAN() if not available).

```
with cte as (  
  select  
    o.order_id,  
    o.order_date,  
    oi.total_price,  
    row_number() over(order by oi.total_price) as row_num_asc,  
    row_number() over(order by oi.total_price desc) as row_num_desc  
  from orders o  
  join order_items oi on o.order_id=oi.order_id  
)  
select avg(total_price)  
from cte  
where abs(cast(row_num_asc as signed) - cast(row_num_desc as signed)) <= 1;
```

92. Find customers who have purchased from at least 3 different categories but have never purchased from the 'electronics' category.

```
select  
  concat(c.first_name, ' ', c.last_name) as full_name,  
  count(distinct ca.category_id) as category_count  
from orders o  
  join order_items oi on o.order_id = oi.order_id  
  join products p on p.product_id = oi.product_id  
  join category ca on p.category_id = ca.category_id  
  join customers c on o.customer_id = c.customer_id  
where ca.category_name != 'Electronics'  
group by concat(c.first_name, ' ', c.last_name)  
having count(distinct ca.category_id) >= 3;
```

93. Identify the most significant drop ($\geq 20\%$) in monthly sales for any product compared to the previous month.

```
WITH cte AS (  
    SELECT  
        p.product_name,  
        YEAR(o.order_date) AS year_,  
        MONTH(o.order_date) AS month_,  
        SUM(oi.total_price) AS current_sales  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN products p ON oi.product_id = p.product_id  
    GROUP BY p.product_name, YEAR(o.order_date), MONTH(o.order_date)  
) ,  
cte2 AS (  
    SELECT  
        product_name, year_, month_, current_sales,  
        LAG(current_sales, 1) OVER (PARTITION BY product_name ORDER BY year_, month_) AS  
previous_sales  
    FROM cte  
) ,  
cte3 AS (  
    SELECT  
        product_name, year_, month_, current_sales, previous_sales,  
        ROUND(((current_sales - previous_sales) / previous_sales) * 100, 2) AS growth_ratio  
    FROM cte2  
    WHERE previous_sales IS NOT NULL  
)  
SELECT  
    product_name, year_, month_, current_sales, previous_sales, growth_ratio  
FROM cte3  
WHERE growth_ratio <= -20  
ORDER BY product_name, year_, month_;
```

94.Create a complete sales report with a single query that aggregates revenue, units sold, average order value, and number of customers by category, state, and month.

```
SELECT
    ca.category_name,
    c.state,
    YEAR(o.order_date) AS year_,
    MONTH(o.order_date) AS month_,
    SUM(oi.total_price) AS total_revenue,
    SUM(oi.quantity) AS units_sold,
    ROUND(SUM(oi.total_price) / COUNT(DISTINCT o.order_id), 2) AS avg_order_value,
    COUNT(DISTINCT c.customer_id) AS num_customers
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
JOIN category ca ON p.category_id = ca.category_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY
    ca.category_name,c.state,YEAR(o.order_date),MONTH(o.order_date)
ORDER BY
    year_, month_, ca.category_name, c.state;
```

95. Find sellers whose sales in the last quarter are down by more than 15% compared to the previous quarter.

```
WITH seller_quarterly_sales AS (  
    SELECT  
        s.seller_id,  
        s.seller_name,  
        YEAR(o.order_date) AS year_,  
        QUARTER(o.order_date) AS quarter_,  
        SUM(oi.total_price) AS total_sales  
    FROM seller s  
    JOIN orders o ON s.seller_id = o.seller_id  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY s.seller_id, s.seller_name, YEAR(o.order_date), QUARTER(o.order_date)  
),  
sales_comparison AS (  
    SELECT  
        seller_id, seller_name, year_, quarter_, total_sales AS current_sales,  
        LAG(total_sales, 1) OVER (  
            PARTITION BY seller_id ORDER BY year_, quarter_  
        ) AS prev_sales  
    FROM seller_quarterly_sales  
)  
SELECT  
    seller_name, year_, quarter_, current_sales, prev_sales,  
    ROUND(((prev_sales - current_sales) / prev_sales) * 100, 2) AS growth_percentage  
FROM sales_comparison  
WHERE prev_sales IS NOT NULL  
    AND ((prev_sales - current_sales) / prev_sales) * 100 > 15  
ORDER BY year_, quarter_, seller_name;
```

96.Assign each customer to a tier ('Platinum', 'Gold', 'Silver') based on their total spending percentile (Top 5%, Next 15%, Next 30%).

```
WITH customer_spending AS (  
  SELECT  
    c.customer_id,  
    CONCAT(c.first_name, ' ', c.last_name) AS full_name,  
    SUM(oi.total_price) AS total_spent  
  FROM customers c  
  JOIN orders o ON c.customer_id = o.customer_id  
  JOIN order_items oi ON o.order_id = oi.order_id  
  GROUP BY c.customer_id, full_name  
)  
,  
ranked AS (  
  SELECT  
    customer_id,full_name,  
    total_spent,  
    NTILE(100) OVER (ORDER BY total_spent DESC) AS percentile_rank  
  FROM customer_spending  
)  
SELECT  
  customer_id, full_name,  
  total_spent,  
  CASE  
    WHEN percentile_rank <= 5 THEN 'Platinum'  
    WHEN percentile_rank <= 20 THEN 'Gold'  
    WHEN percentile_rank <= 50 THEN 'Silver'  
    ELSE 'Others'  
  END AS customer_tier  
FROM ranked  
ORDER BY total_spent DESC;
```

97. Calculate the correlation between the day of the week and the average order value.

```
WITH cte AS (  
    SELECT  
        DAYNAME(o.order_date) AS day_of_week,  
        COUNT(DISTINCT o.order_id) AS total_orders,  
        SUM(oi.total_price) AS total_day_sales  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    GROUP BY DAYNAME(o.order_date)  
)  
SELECT  
    day_of_week,  
    total_orders,  
    total_day_sales,  
    ROUND(total_day_sales / total_orders, 2) AS avg_order_value  
FROM cte;
```

98. Executive Summary Query: Create a single, comprehensive query that provides a high-level dashboard with: Total Revenue, Total Orders, Average Order Value, Top Selling Category, Top Performing State, and Customer Count for the current year (2024).

```
WITH base AS (  
    SELECT  
        o.order_id,  
        o.customer_id,  
        c.state,  
        oi.total_price,  
        oi.quantity,  
        ca.category_name  
    FROM orders o  
    JOIN order_items oi ON o.order_id = oi.order_id  
    JOIN customers c ON o.customer_id = c.customer_id  
    JOIN products p ON oi.product_id = p.product_id
```

```
JOIN category ca ON p.category_id = ca.category_id

WHERE YEAR(o.order_date) = 2024

),agg AS (

SELECT

    SUM(total_price) AS total_revenue,

    COUNT(DISTINCT order_id) AS total_orders,

    COUNT(DISTINCT customer_id) AS customer_count,

    ROUND(SUM(total_price) / COUNT(DISTINCT order_id), 2) AS avg_order_value

FROM base

),top_category AS (

SELECT category_name

FROM base

GROUP BY category_name

ORDER BY SUM(total_price) DESC

LIMIT 1

),top_state AS (

SELECT state

FROM base

GROUP BY state

ORDER BY SUM(total_price) DESC

LIMIT 1

)

SELECT

    a.total_revenue, a.total_orders,

    a.customer_count,

    a.avg_order_value,

    tc.category_name AS top_selling_category,

    ts.state AS top_performing_state

FROM agg a

CROSS JOIN top_category tc

CROSS JOIN top_state ts;
```